



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DESIGN OF A SMART-CITY EMERGENCY VEHICLE ROUTING SYSTEM (EVRS) WITH 100+ INTERSECTIONS (NODES) AND 200+ WEIGHTED ROADS (EDGES AND WEIGHTS)

ASSIGNMENT REPORT

*Submitted in partial fulfilment of the requirements
for the course*

CSA0307 – DATA STRUCTURES

towards the award of the degree of
BACHELOR OF ENGINEERING
IN

B.TECH IN AIDS /IT

Submitted by

Monica Sentamil K(192524315)

Madhumitha V (192521332)

Mahalakshmi S (192524372)

Under the Supervision of

Dr. K. Kiruthika

**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
SEPTEMBER 2026**

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Artificial Intelligence and Data Science /Information technology
Programme:	Bachelor of Technology
Course Code & Course Name	CSA0311 Data Structures
Academic Year / Batch	2025 Batch
Faculty Name	Dr.Kiruthika
Assignment Title	Design of a smart-city emergency vehicle routing system (evrs) with 100+ intersections (nodes) and 200+ weighted roads (edges and weights)
Date of Issue	31/August/2026
Date of Submission	2/September/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data type for construction of the smart city emergency system. CO4: Make use of heaps, priority queue and hashing techniques for graph storage and route cache for repeated queries in C. CO5: Develop robust smart city emergency system by implementing shortest path algorithms such as Dijkstra's algorithm, Floyd-Warshall.
Bloom's Taxonomy Level	L4 – Analyze
SDG Mapping (SDG 1-17)	SDG 9 - Industry, Innovation, and Infrastructure SDG 11 – Sustainable Cities and Communities;
Industry / Societal Relevance	Industry

TABLE OF CONTENTS

S.No.	Contents	Page No.
1.	Problem Statement	3
2.	Objective	4
3.	Requirements and Environment Used	5
4.	Design / Proposed Solution	7
5.	Algorithm / Pseudocode / Flowchart	11
6.	Implementation / Source Code	15
7.	Test Cases and Expected/Actual Results	19
8.	Execution Screenshots / Output	21
9.	Analysis and Discussion	24
10.	Conclusion	27
11.	Individual Contribution of Group Members	28
12.	References	29
13.	One Page Write-Up	30

1. PROBLEM STATEMENT

Emergency medical services and fire-rescue operations are fundamentally time-critical. Clinical studies on cardiac arrest and major trauma consistently show that survival probability falls sharply for every minute of delay in the arrival of an ambulance or a fire engine, and international emergency-response standards (e.g., the widely cited 8-minute response benchmark used by many fire-and-rescue authorities) treat travel time from depot to incident as the single most controllable variable in the entire emergency-response chain. In a modern metropolitan area, however, the road network that emergency vehicles must traverse is neither static nor uniform: traffic congestion fluctuates by the minute, accidents and waterlogging close roads without notice, weather conditions change the effective speed of travel, and the sheer scale of the city — hundreds of intersections and road segments — makes manual or ad-hoc route selection unreliable and slow.

Traditional static/offline mapping systems assume that the underlying road network barely changes and that the same shortest route remains valid for extended periods. This assumption breaks down catastrophically during rush hours, natural disasters, or large public events, when the fastest route for an ambulance five minutes ago may now be blocked or gridlocked. Recomputing an entire routing table from scratch after every such change is computationally expensive and cannot meet the strict sub-second decision latency that a life-critical dispatch system demands.

This project addresses the following concrete engineering problem: design and implement a smart-city Emergency Vehicle Routing System (EVRS) that (a) models a realistic city road network containing more than 100 intersections (graph nodes) and more than 200 weighted, directional roads (graph edges), (b) supports real-time, $O(1)$ -time mutation of traffic conditions, road blockages, weather, and signal-preemption (“green corridor”) state without rebuilding the graph, (c) computes the fastest emergency route between any two points of the city — or from a set of candidate depots to an incident location — well within a 200-millisecond service-level agreement (SLA), and (d) guarantees that the computed route is provably optimal with respect to the chosen cost criterion (travel time or physical distance), while remaining verifiable through automated tests and quantitative benchmarking.

In short, the problem is a dynamic shortest-path problem on a large, weighted, directed graph under continuously changing edge weights, solved under a hard real-time latency constraint, and applied to the socially significant domain of emergency medical and fire-rescue dispatch in a smart city.

2. OBJECTIVE

The overall aim of this assignment is to design, implement and empirically validate a shortest-path-based Emergency Vehicle Routing System that a smart-city command centre could use to dispatch ambulances and fire engines along the fastest available route, while dynamically reacting to live traffic and incident data. The specific objectives are:

1. To construct a hash-map-based graph representation of a city road network with at least 100 intersections and 200 weighted road segments, capable of $O(1)$ edge lookup, traffic-multiplier update, and roadblock toggling.
2. To implement Dijkstra's shortest-path algorithm using a binary min-heap priority queue as the baseline, guaranteed-optimal routing algorithm.
3. To implement the A* search algorithm with an admissible and consistent Euclidean-distance-based heuristic, so as to reduce the number of nodes explored relative to Dijkstra while preserving optimality.
4. To implement supplementary routing strategies — an all-pairs Floyd–Warshall router and an unweighted Breadth-First-Search hop-count router — for comparative benchmarking purposes.
5. To model dynamic, real-world operating conditions: live traffic congestion multipliers, road-blocking incidents (accidents, flooding, construction), weather friction factors, signal-preemption “green corridors” for emergency vehicles, and rush-hour surges.
6. To design a multi-depot dispatch module that, given an incident location and a set of candidate hospitals/fire-stations/ambulance depots, identifies the single fastest-responding unit.
7. To expose the entire system through a RESTful Flask web-service layer, supporting route computation, live search-trace visualisation, traffic mutation, and on-demand benchmarking.
8. To empirically benchmark Dijkstra versus A* over a large number of randomised queries, verify that both consistently satisfy a sub-200 ms latency SLA and produce cost-identical (optimal) paths, and quantitatively demonstrate the search-space reduction achieved by the heuristic.
9. To validate the complete system through an automated unit- and integration-test suite covering graph construction, algorithmic optimality, latency, dynamic updates, rerouting around blockages, multi-depot dispatch, and REST API correctness.

3. REQUIREMENTS AND ENVIRONMENT USED

3.1 Functional Requirements

- The system shall represent a city as a directed, weighted graph of intersections (nodes) and road segments (edges), each edge carrying distance, speed limit, road type, lane count and a live traffic multiplier.
- The system shall allow $O(1)$ mutation of an edge's traffic multiplier, blocked/unblocked status, and incident description at run time.
- The system shall compute the shortest / fastest path between any two valid intersection identifiers for a given vehicle type (ambulance, fire engine, or normal traffic) and cost criterion (time or distance).
- The system shall differentiate travel-time computation for emergency vehicles by modelling siren pre-emption, dedicated emergency lanes, and green-corridor signal bypass.
- The system shall support multi-depot dispatch: given an incident node and a list of candidate depot nodes, it shall return the fastest responding unit ranked against all candidates.
- The system shall provide step-by-step search-trace data (frontier, visited set, relaxed edges) suitable for live visualisation of the algorithm's execution.
- The system shall expose all the above functionality via a REST API (Flask) consumable by a web dashboard.
- The system shall provide a benchmarking utility capable of running N randomised source–destination queries and reporting execution-time, nodes-explored, heap-operations and memory statistics for each algorithm.

3.2 Non-Functional Requirements

- Performance: every point-to-point route query must complete within a 200 ms Service-Level Agreement (target observed: well under 1 ms on the reference network).
- Scalability: the graph representation must remain $O(1)$ for traffic/roadblock mutation regardless of network size, since real cities may contain thousands of intersections.
- Correctness: Dijkstra and A* must always return routes of identical optimal cost (within floating-point tolerance) for the same query, since A* is only useful if it does not sacrifice optimality.
- Reliability: the routing engine must gracefully handle disconnected graphs, blocked roads, and invalid node identifiers without crashing.

- **Maintainability:** the codebase is modularised into independent components (graph model, algorithms, traffic simulation, network generator, web layer) to allow isolated testing and extension.
- **Testability:** every core behaviour must be covered by an automated test in the unittest test-suite.

3.3 Hardware Environment

- **Processor:** any modern x86-64 / ARM64 processor (Intel Core i5 / AMD Ryzen 5 or equivalent, 4 cores or more recommended for the Flask development server and benchmark runs).
- **Memory:** minimum 4 GB RAM (the 113-node / 454-edge reference network and all in-memory routing tables occupy only a few hundred kilobytes).
- **Storage:** negligible (< 50 MB including all dependencies) — no persistent database is required since the network is generated procedurally at start-up.

3.4 Software Environment

- **Programming Language:** Python 3.12
- **Web Framework:** Flask 3.x (REST API layer, template rendering)
- **Core Libraries:** heapq (binary min-heap for Dijkstra / A*), collections.deque (BFS queue), dataclasses, math, statistics
- **Benchmarking & Visualisation:** matplotlib (headless “Agg” backend) for generating comparative performance charts
- **Testing Framework:** Python's built-in unittest module, including Flask's test_client for REST API integration tests
- **Front-End (dashboard consumption layer):** HTML5 / CSS3 / JavaScript served through Flask's Jinja2 template engine and static file handler
- **Development Tools:** Visual Studio Code, Git for version control, Python virtual environment (venv) for dependency isolation
- **Operating System:** cross-platform — developed and tested on Linux (Ubuntu) and verified to be portable to Windows/macOS

3.5 Modern Tool Usage

In line with the CSE/IT tool guidance of the course, this project makes explicit use of Python as the implementation language, the heapq module as a modern, well-tested priority-queue implementation (avoiding a hand-rolled heap and its associated risk of bugs), Flask as an industry-standard micro web-framework for exposing the algorithm as a service, matplotlib for generating quantitative, reproducible performance graphs, and

Git for iterative, version-controlled development. The automated unittest suite plays the role of a continuous-validation tool, re-verifying correctness and the 200 ms latency SLA on every code change.

4. DESIGN / PROPOSED SOLUTION

4.1 System Architecture Overview

The proposed solution follows a clean, four-layer architecture that separates data modelling, algorithmic logic, dynamic-state simulation, and service exposure. This separation allows each layer to be independently tested, benchmarked, and replaced (for example, swapping the procedurally generated Metropolis network for a real GIS-derived network would require no change to the algorithm or API layers).

Layer	Module(s)	Responsibility
1. Data / Graph Layer	graph.py	Defines Intersection & RoadSegment data models and the CityRoadNetwork hash-map graph, with $O(1)$ traffic/roadblock mutation and dynamic edge-weight computation.
2. Network Generation Layer	generator.py	Procedurally builds the 113-node / 454-directed-edge “Metropolis” city network across 5 districts, plus a configurable ProceduralCityGenerator for arbitrary network sizes.
3. Algorithm Layer	algorithms.py	Implements DijkstraRouter, AStarRouter, FloydWarshallRouter, BFSRouter and MultiDepotDispatcher, all returning a uniform PathResult telemetry object.
4. Dynamic Simulation Layer	traffic_engine.py	Injects/clears incidents, simulates rush-hour surges and stochastic micro-fluctuations in live traffic.
5. Service / Presentation Layer	app.py	Exposes REST endpoints (/api/route, /api/trace, /api/traffic/*, /api/dispatch/multi, /api/benchmark/*) consumed by the command-centre dashboard.
6. Verification Layer	test_routing.py, benchmark_cli.py	Automated correctness tests and a standalone statistical benchmark / chart generator.

4.2 Graph Data Model

The road network is represented as a hash-map graph rather than an adjacency matrix or adjacency list of linked nodes, because the dominant operational requirement is $O(1)$ random-access update of individual edges (traffic surges, roadblocks) rather than iteration over the whole matrix. Two dataclasses form the vertex and edge model:

- Intersection — represents a node (hospital, fire station, ambulance depot, ordinary intersection, or landmark) with an identifier, name, (x, y) coordinates, district, elevation, and a base traffic-signal delay. A distance_to() method returns the Euclidean distance to another intersection, used both for automatic edge-length inference and for the A* heuristic.

- **RoadSegment** — represents a single directed road between two intersections, carrying distance (km), speed limit (km/h), road type (highway / arterial / local / emergency_lane), lane count, a mutable traffic_multiplier, a mutable is_blocked flag, and an incident description. Its calculate_travel_time_seconds() method is the heart of the dynamic cost model.

The graph itself, CityRoadNetwork, stores nodes as a `HashMap[node_id → Intersection]` and edges as a `HashMap[u → HashMap[v → RoadSegment]]` (a “dictionary of dictionaries”). This nested hash-map layout gives $O(1)$ amortised time for: adding an intersection, adding a road, looking up a specific edge `get_edge(u, v)`, updating a traffic multiplier `update_traffic(u, v, ...)`, and toggling a roadblock `set_road_block(u, v, ...)`. Global state such as `weather_condition`, `weather_factor`, `global_congestion_factor` and `green_corridor_active` is also stored on the graph object so that a single $O(1)$ write immediately affects every subsequent weight query across the whole city.

4.3 Dynamic Cost (Edge-Weight) Model

Each call to `RoadSegment.calculate_travel_time_seconds()` derives an effective travel time from several composable factors, so that the same static graph produces different weights depending on who is asking and what the city conditions are:

- Free-flow time — distance divided by posted speed limit, converted to seconds.
- Traffic multiplier — a live congestion factor (1.0 = clear, up to ~5.0 = gridlock).
- Vehicle-type privilege — ambulances and fire engines suffer only 60% of the congestion penalty under normal conditions, only 25% of the penalty when a green corridor is active, and are treated as fully clear on dedicated emergency_lane roads; ordinary vehicles bear the full multiplier.
- Weather friction — a city-wide multiplier (Clear = 1.00, Light Rain = 1.15, Heavy Rain/Flood = 1.45, Fog = 1.30, Storm = 1.60).
- Road-tier turning-friction penalty — highways (0.95×) are slightly faster per kilometre than local streets (1.15×) due to fewer turns and signals.
- Blockage — a blocked road returns an infinite weight, effectively removing that edge from the search space without deleting it from the graph (so it can be $O(1)$ re-opened later).

4.4 The Reference City Network (“Metropolis”)

To satisfy the ≥ 100 -node / ≥ 200 -edge requirement with a realistic, human-interpretable layout, `generator.py` procedurally constructs a 5-district Metropolis network. When executed, the generator produces 113 intersections and 454 directed road segments (227 bidirectional physical roads), comfortably exceeding both minimums. The composition of node types and districts is summarised below.

District	Node Count	Character
Medical District (West)	24	4 hospitals, 3 ambulance depots, clinics and local junctions
Downtown Core (Central)	25	Civic centre, commercial junctions, dense arterial mesh
Industrial Zone (South)	24	Factories, warehouses, fire stations, freight corridors
Suburbs (North)	20	Residential streets, local roads, lower speed limits
Airport & Tech Corridor (East)	20	Highway ring-road access, tech campuses, express lanes

Node Type	Count	Role
Intersection	83	Ordinary road junctions used for path continuity
Landmark	11	Notable named locations (parks, plazas, campuses)
Depot	8	Ambulance stations
Hospital	6	Emergency / trauma admission points
Fire Station	5	Fire-engine dispatch points

The network additionally includes a high-speed perimeter ring road and central arterial avenues connecting the districts, plus dedicated emergency-express corridors and secondary local connectors that keep the mesh fully connected, ensuring every intersection is reachable from every other intersection under normal (non-blocked) conditions.

4.5 Dynamic Traffic Simulation Design

The `DynamicTrafficEngine` class wraps a `CityRoadNetwork` and provides the operational controls a smart-city command centre would need: `inject_incident()` (block a road or apply a heavy congestion surge and log the event), `clear_incident()` / `clear_all_incidents()` ($O(1)$ restoration), `simulate_rush_hour_surge()` (raises multipliers on downtown/arterial/local roads more than on highways, mimicking real commuter patterns), `reset_to_free_flow()` (returns the whole city to multiplier = 1.0), and `random_stochastic_fluctuation()` (a small random walk applied to every edge's multiplier each “tick”, with a 10% chance of spawning a minor stalled-vehicle incident) to emulate continuous, organic traffic telemetry rather than static conditions.

4.6 Route-Computation Data Flow

At query time the following data flow takes place, whether triggered from the REST API or the benchmark CLI:

1. A client (dashboard or test harness) submits a start node, target node, vehicle type and cost criterion.
2. The chosen router (Dijkstra, A*, Floyd–Warshall or BFS) queries the graph via `get_neighbors()` / `get_weight()`, which internally applies the live traffic multiplier, weather factor, global congestion factor and green-corridor state.
3. The algorithm explores the graph until the target is settled (Dijkstra/A*) or the full distance matrix is available (Floyd–Warshall), producing a parent-pointer chain.
4. The path is reconstructed by walking parent pointers from target back to source, then reversed.
5. Total distance and total travel time are recomputed edge-by-edge along the final path (for reporting accuracy, independent of the internal cost accumulator).
6. A `PathResult` telemetry object — algorithm name, path, cost, nodes explored, heap operations, execution time, memory estimate and SLA-compliance flag — is returned and serialised to JSON for the caller.

This uniform `PathResult` contract across all four routers is what makes apples-to-apples benchmarking possible in Section 9 of this report.

5. ALGORITHM / PSEUDOCODE / FLOWCHART

5.1 Dijkstra's Algorithm (Min-Heap Priority Queue)

Dijkstra's algorithm is used as the guaranteed-optimal baseline router. It is implemented with Python's `heapq` binary min-heap, a `HashMap` distance table, a `HashMap` parent table for path reconstruction, and a `HashSet` visited table for $O(1)$ membership tests. Its time complexity is $O((V + E) \log V)$ and its space complexity is $O(V)$. The algorithm follows the classical formulation of Dijkstra [2].

```

ALGORITHM Dijkstra(graph, source, target, vehicleType, criterion):
    priorityQueue <- MIN-HEAP()
    PUSH(priorityQueue, (0, source))
    distance[source] <- 0
    parent[source] <- NULL
    visited <- EMPTY-SET()

    WHILE priorityQueue is NOT EMPTY:
        (currentDist, u) <- POP-MIN(priorityQueue)
        IF u IN visited: CONTINUE
        ADD u TO visited
        IF u == target: BREAK      // early exit -> optimal distance found

        FOR EACH (v, road) IN graph.neighbors(u):
            IF v IN visited OR road.isBlocked: CONTINUE
            weight <- graph.getWeight(u, v, vehicleType, criterion)
            IF weight == INFINITY: CONTINUE
            newDist <- currentDist + weight
            IF newDist < distance.get(v, INFINITY):
                distance[v] <- newDist
                parent[v] <- u
                PUSH(priorityQueue, (newDist, v))

    RETURN reconstructPath(parent, target), distance[target]

```

Because entries are lazily invalidated (a node popped after it is already visited is simply skipped) rather than decrease-keyed in place, the implementation avoids the complexity of a decrease-key heap operation while retaining correct $O((V+E) \log V)$ behaviour — a standard, well-understood trade-off in Dijkstra implementations built on Python's `heapq`.

5.2 A* Search Algorithm (Admissible Heuristic)

A* extends Dijkstra by ordering the priority queue on $f(n) = g(n) + h(n)$, where $g(n)$ is the exact accumulated cost from the source to node n , and $h(n)$ is an admissible, consistent estimate of the remaining cost from n to the target. The heuristic uses the straight-line Euclidean distance between the node's coordinates and the target's coordinates. For the time criterion, the heuristic uses a conservative lower-bound travel time derived from the model's stated maximum posted speed of 100 km/h and the minimum road-tier factor of 0.85, giving an optimistic effective speed of approximately 117.65 km/h. For the distance

criterion, the Euclidean distance is used directly. This construction keeps the heuristic optimistic with respect to the implemented edge-cost model and therefore preserves A* optimality. The A* formulation follows Hart, Nilsson and Raphael [3].

```

FUNCTION heuristic(node, target, criterion):
  d <- EuclideanDistance(node.xy, target.xy)    // straight-line km
  IF criterion == "distance": RETURN d
  RETURN (d / MAX_POSSIBLE_SPEED_KMH) * 3600    // optimistic time,
seconds

ALGORITHM AStar(graph, source, target, vehicleType, criterion):
  priorityQueue <- MIN-HEAP()
  PUSH(priorityQueue, (heuristic(source,target), 0, source))
  gScore[source] <- 0
  parent[source] <- NULL
  visited <- EMPTY-SET()

  WHILE priorityQueue is NOT EMPTY:
    (f, g, u) <- POP-MIN(priorityQueue)
    IF u IN visited: CONTINUE
    ADD u TO visited
    IF u == target: BREAK

    FOR EACH (v, road) IN graph.neighbors(u):
      IF v IN visited OR road.isBlocked: CONTINUE
      weight <- graph.getWeight(u, v, vehicleType, criterion)
      IF weight == INFINITY: CONTINUE
      newG <- g + weight
      IF newG < gScore.get(v, INFINITY):
        gScore[v] <- newG
        parent[v] <- u
        f <- newG + heuristic(v, target, criterion)
        PUSH(priorityQueue, (f, newG, v))

  RETURN reconstructPath(parent, target), gScore[target]

```

Proof sketch of admissibility and consistency

For the distance criterion, the straight-line Euclidean distance is never greater than the corresponding road-network distance by the triangle inequality. For the time criterion, the heuristic uses 117.65 km/h, which is deliberately derived from the fastest modeled combination (100 km/h maximum posted speed with the 0.85 emergency-lane tier factor). Because traffic and weather factors in the implemented model are never below 1.0, and the remaining vehicle-specific factors do not reduce the cost below this modeled lower bound, the heuristic cannot overestimate the true remaining travel time. The Euclidean distance function also satisfies the triangle inequality, so the resulting heuristic is

consistent for the modeled non-negative edge costs. An admissible and consistent heuristic guarantees that A* terminates with the true shortest-path cost when the target is popped from the priority queue. This theoretical property is subsequently checked empirically through the 150-query benchmark, where Dijkstra and A* produced cost-identical results for every query.

5.3 Floyd–Warshall All-Pairs Shortest Path (Comparative Router)

For scenarios in which many repeated queries are expected against a relatively static graph, the system additionally offers a Floyd–Warshall all-pairs pre-computation, implemented as an $O(V^3)$ dynamic-programming relaxation over a distance matrix, with an accompanying “next-hop” matrix for $O(\text{path length})$ reconstruction. Time complexity is $O(V^3)$ to build the table once, after which any query is answered in $O(1)$ matrix lookup plus $O(\text{path length})$ reconstruction.

```

ALGORITHM FloydWarshall(graph):
  FOR i, j IN nodes: dist[i][j] <- weight(i,j) if edge exists else INFINITY
  FOR k IN nodes:
    FOR i IN nodes:
      FOR j IN nodes:
        IF dist[i][k] + dist[k][j] < dist[i][j]:
          dist[i][j] <- dist[i][k] + dist[k][j]
          next[i][j] <- next[i][k]
  RETURN dist, next

```

5.4 Breadth-First Search (Unweighted Hop-Count Router)

A Breadth-First-Search router is included to represent the naive baseline of “fewest intersections crossed”, ignoring distance and travel time altogether. It runs in $O(V + E)$ using a FIFO queue and is useful only as a contrast case in Section 9 to demonstrate why hop-count is a poor proxy for real emergency-response time.

```

ALGORITHM BFS(graph, source, target):
  queue <- FIFO-QUEUE([source]); visited <- {source}; hops[source] <- 0
  WHILE queue is NOT EMPTY:
    u <- DEQUEUE(queue)
    IF u == target: BREAK
    FOR EACH (v, road) IN graph.neighbors(u):
      IF road.isBlocked OR v IN visited: CONTINUE
      visited <- visited UNION {v}; parent[v] <- u; hops[v] <- hops[u] + 1
      ENQUEUE(queue, v)
  RETURN reconstructPath(parent, target), hops[target]

```

5.5 Multi-Depot Emergency Dispatch

The many-to-one dispatch problem — “which of my N ambulances/fire-engines can reach this incident fastest?” — is solved by running the chosen single-source router (Dijkstra or A*) once per candidate depot and selecting the minimum-travel-time result.

This is $O(N \times \text{routing-cost})$ but N (typically 3–8 depots) is small, so the wall-clock cost remains sub-millisecond in practice.

```

ALGORITHM MultiDepotDispatch(graph, depotList, incidentNode, vehicleType):
  results <- []
  FOR EACH depot IN depotList:
    r <- FindShortestPath(graph, depot, incidentNode, vehicleType, "time")
    IF r.found: APPEND results, r
  SORT results BY travelTimeSeconds ASCENDING
  RETURN results[0] // fastest responding unit, plus full ranked list

```

5.6 Flowchart — End-to-End Route Computation Pipeline

The overall request-to-response pipeline (identical for both Dijkstra and A*) can be represented as the following sequential flow:

1. START — client sends {start, target, vehicleType, criterion}.
2. Validate start and target exist in the graph. If not → return HTTP 400 error, STOP.
3. Initialise priority queue with the source node at cost 0 (plus heuristic, for A*).
4. Decision: is the priority queue empty? → YES: no route exists, return found = false, STOP.
5. Decision: is the priority queue empty? → NO: pop the minimum-cost node u .
6. Decision: has u already been visited? → YES: discard and go back to step 4.
7. Mark u visited. Decision: is u the target? → YES: go to step 10 (path found).
8. NO: for every unblocked, unvisited neighbour v of u , compute the dynamic edge weight (traffic $\times$ weather $\times$ vehicle-privilege $\times$ road-tier) and relax the distance if improved, pushing v back onto the queue.
9. Return to step 4 (loop).
10. Reconstruct the path by following parent pointers from target to source, then reverse it.
11. Recompute exact total distance and total time along the reconstructed path.
12. Package the path, cost, nodes-explored, heap-operations, execution-time and SLA-compliance flag into a PathResult and return it to the caller. END.

6. IMPLEMENTATION / SOURCE CODE

The complete system is implemented in Python 3.12 across seven cooperating modules. This section reproduces the most significant excerpts of each module; the full source listings were submitted alongside this report as separate .py files.

6.1 Module Structure

File	Lines (approx.)	Purpose
graph.py	~300	Intersection & RoadSegment dataclasses; CityRoadNetwork hash-map graph with O(1) mutation.
generator.py	~330	build_metropolis_network() (113 nodes / 454 edges) and ProceduralCityGenerator for arbitrary sizes.
algorithms.py	~955	DijkstraRouter, AStarRouter, FloydWarshallRouter, BFSRouter, MultiDepotDispatcher, PathResult, AlgorithmStep.
traffic_engine.py	~140	DynamicTrafficEngine: incident injection/clearing, rush-hour surge, stochastic fluctuation.
app.py	~300	Flask REST API: routing, tracing, traffic control, dispatch, benchmarking endpoints.
benchmark_cli.py	~235	Standalone CLI: 150-iteration statistical benchmark, SLA verification, matplotlib chart generation.
test_routing.py	~130	unittest suite: 7 automated test cases covering the whole system end-to-end.

6.2 Graph Model — graph.py

The RoadSegment.calculate_travel_time_seconds() method is the single most important piece of business logic in the project, since every routing decision ultimately depends on it:

```
def calculate_travel_time_seconds(self, vehicle_type="ambulance",
                                weather_factor=1.0,
                                green_corridor_active=False):
    if self.is_blocked:
        return float('inf')
    base_time = self.free_flow_time_seconds
    effective_traffic = self.traffic_multiplier
    if vehicle_type in ("ambulance", "fire_engine"):
        if self.road_type == "emergency_lane":
            effective_traffic = 1.0
        elif green_corridor_active:
            effective_traffic = max(1.0, 1.0 + (self.traffic_multiplier-1.0)*0.25)
        else:
            effective_traffic = max(1.0, 1.0 + (self.traffic_multiplier-1.0)*0.6)
    total_time = base_time * effective_traffic * weather_factor
```

```

tier_penalty = {"highway": 0.95, "arterial": 1.0,
                "local": 1.15, "emergency_lane": 0.85}.get(self.road_type, 1.0)
return total_time * tier_penalty

```

The CityRoadNetwork class stores the graph as nested HashMaps and offers $O(1)$ mutation primitives, e.g.:

```

def update_traffic(self, u, v, traffic_multiplier, bidirectional=True):
    updated = False
    if u in self.adjacency and v in self.adjacency[u]:
        self.adjacency[u][v].traffic_multiplier = max(1.0, float(traffic_multiplier))
        self.adjacency[u][v].last_updated = time.time()
        updated = True
    if bidirectional and v in self.adjacency and u in self.adjacency[v]:
        self.adjacency[v][u].traffic_multiplier = max(1.0, float(traffic_multiplier))
        updated = True
    return updated #  $O(1)$  — direct HashMap key access, no traversal required

```

6.3 Routing Algorithms — algorithms.py

Excerpt of the Dijkstra core loop (min-heap based, lazy-deletion visited check):

```

priority_queue = []
heapq.heappush(priority_queue, (0.0, start_id))
distances = {start_id: 0.0}
parents = {start_id: None}
visited = set()
while priority_queue:
    current_dist, current_node = heapq.heappop(priority_queue)
    if current_node in visited:
        continue
    visited.add(current_node)
    if current_node == target_id:
        break # optimal distance guaranteed on first pop
    for neighbor_id, road in self.network.get_neighbors(current_node).items():
        if neighbor_id in visited or road.is_blocked:
            continue
        edge_weight = self.network.get_weight(current_node, neighbor_id,
                                              vehicle_type, criterion)
        if math.isinf(edge_weight):
            continue
        new_dist = current_dist + edge_weight
        if new_dist < distances.get(neighbor_id, float('inf')):
            distances[neighbor_id] = new_dist
            parents[neighbor_id] = current_node
            heapq.heappush(priority_queue, (new_dist, neighbor_id))

```

Excerpt of the A* admissible heuristic function:

```

def calculate_heuristic(self, node_id, target_id, criterion="time"):
    u = self.network.nodes.get(node_id)
    v = self.network.nodes.get(target_id)
    euclidean_dist = u.distance_to(v)

```

```

if criterion == "distance":
    return euclidean_dist
max_possible_speed_kmh = 100.0
return (euclidean_dist / max_possible_speed_kmh) * 3600.0 # admissible, consistent

```

6.4 Dynamic Traffic Simulation — traffic_engine.py

```

def inject_incident(self, u, v, incident_type="Major Multi-Vehicle Accident",
                    block_road=True, traffic_surge=4.5):
    incident_record = {
        "id": f"INC-{int(time.time()*1000)%100000}", "u": u, "v": v,
        "type": incident_type, "blocked": block_road,
        "multiplier": traffic_surge, "timestamp": time.time()
    }
    if block_road:
        self.network.set_road_block(u, v, is_blocked=True,
                                    incident_description=incident_type, bidirectional=True)
    else:
        self.network.update_traffic(u, v, traffic_multiplier=traffic_surge, bidirectional=True)
    self.active_incidents.append(incident_record)
    return incident_record

```

6.5 REST API Layer — app.py

The Flask layer exposes each capability of the system as a stateless JSON endpoint. The principal routes are:

Method & Route	Purpose
GET /api/network	Returns the full serialised graph (nodes, edges, live traffic state).
POST /api/route	Computes the route with Dijkstra, A*, Floyd–Warshall and/or BFS and returns a comparison block.
POST /api/trace	Returns step-by-step search-frontier snapshots for live UI animation.
POST /api/traffic/update	O(1) update of a road's live traffic multiplier.
POST /api/traffic/block	Injects or clears a roadblock / incident.
POST /api/traffic/rush_hour	Triggers a city-wide rush-hour congestion surge.
POST /api/traffic/reset	Resets the whole city to free-flow conditions.
POST /api/dispatch/multi	Multi-depot fastest-unit dispatch for an incident.
POST /api/controls/settings	Updates weather, green-corridor and global congestion settings.
POST /api/benchmark/live	Runs an on-demand N-iteration benchmark and returns statistics.

```

@app.route("/api/route", methods=["POST"])
def compute_route():
    payload = request.get_json() or {}
    start_id = payload.get("start", "M01")
    target_id = payload.get("target", "D19")
    vehicle_type = payload.get("vehicle_type", "ambulance")
    criterion = payload.get("criterion", "time")
    ...
    res_d = dijkstra_router.find_shortest_path(start_id, target_id, vehicle_type, criterion)
    res_a = astar_router.find_shortest_path(start_id, target_id, vehicle_type, criterion)
    response_data["comparison"] = {
        "node_savings_percentage": round(savings_pct, 1),
        "speed_ratio": round(d_res["execution_time_ms"] / a_res["execution_time_ms"], 2),
        "cost_identical": abs(d_res["total_cost"] - a_res["total_cost"]) < 1e-3,
        "both_sla_compliant": d_res["sla_met"] and a_res["sla_met"]
    }
    return jsonify({"status": "success", "data": response_data})

```

6.6 Network Generator — generator.py

```

def build_metropolis_network() -> CityRoadNetwork:
    net = CityRoadNetwork(name="Metropolis-SmartCity")
    medical_nodes = [
        ("M01", "Trauma Center H1 (Metro General)", 12.0, 22.0, "hospital"),
        ("M05", "Ambulance Station Alpha (AMB-1)", 18.0, 22.0, "depot"),
        # ... 5 districts, 113 intersections in total ...
    ]
    for nid, name, x, y, ntype in medical_nodes:
        net.add_intersection(nid, name, x, y, node_type=ntype, district="Medical District")
    # Perimeter ring road, arterial avenues, emergency express corridors,
    # and secondary local connectors are added next, yielding
    # 113 nodes and 454 directed road segments in the final graph.
    return net

```

7. TEST CASES AND EXPECTED/ACTUAL RESULTS

The system was validated using an automated unittest suite (`test_routing.py`) containing seven end-to-end test cases. Every test was executed against the live 113-node / 454-edge Metropolis network and the full Flask application (via Flask's `test_client`), and every test passed. The table below summarises each test case together with its expected and observed outcome.

#	Test Case	Input / Action	Expected Result	Actual Result
TC-01	Network scale constraint	Build the Metropolis network and inspect node/edge counts	<code>num_nodes > 100</code> and <code>num_edges > 200</code>	113 intersections, 454 directed roads — PASS
TC-02	Algorithm optimality equivalence	Run Dijkstra and A* on 5 fixed source–target pairs (e.g. M01→D19, I01→T10)	Dijkstra and A* return identical <code>total_cost</code> ($\Delta < 1e-3$)	All 5 pairs matched exactly — PASS
TC-03	Sub-200 ms latency SLA	Run 25 repeated route queries (M01→T10) with Dijkstra and A*	<code>execution_time_ms < 200</code> for every run; <code>sla_met == True</code>	Max observed ≈ 0.34 ms; all runs compliant — PASS
TC-04	Dynamic traffic update ($O(1)$)	Inject <code>traffic_multiplier = 5.0</code> on the first edge of an existing route, then re-query	New route <code>total_time_seconds</code> strictly greater than before the update	Route cost increased as expected — PASS
TC-05	Roadblock & automatic re-routing	Block the first edge on the M01→D19 route, then re-query	A valid alternative multi-hop path is returned that excludes the blocked edge	Bypass route found; blocked edge absent from new path — PASS
TC-06	Multi-depot dispatch	Dispatch query with 4 candidate depots (M01, M06, I01, R05) to incident T20	<code>best_unit</code> is not null; <code>sla_met</code> is True; all 4 candidates evaluated	Fastest unit correctly identified; 4/4 candidates ranked — PASS
TC-07	Flask REST API endpoints	Exercise <code>/api/network</code> , <code>/api/route</code> , <code>/api/trace</code> and <code>/api/traffic/update</code>	HTTP 200 on every call; response JSON contains the expected keys (dijkstra, astar, comparison, etc.)	All four endpoints returned HTTP 200 with well-formed JSON — PASS

Executing `python3 -m unittest test_routing -v` produced the following console output, confirming a 100% pass rate across all seven cases in under 0.04 seconds of wall-clock time:

```

test_algorithm_optimality_equivalence ... ok
test_dynamic_traffic_update_o1 ..... ok
test_flask_rest_api_endpoints ..... ok
test_multi_depot_dispatch ..... ok
test_network_scale_constraints ..... ok
[Test] Intersections: 113, Directed Roads: 454
test_roadblock_and_rerouting ..... ok
test_sub_200ms_latency_sla ..... ok

```

```

-----
Ran 7 tests in 0.037s

```

```

OK

```

7.1 Additional Statistical Validation (150-Query Randomised Benchmark)

Beyond the seven discrete unit tests, `benchmark_cli.py` performs a much larger statistical validation: 150 randomly generated source–destination pairs (seeded for reproducibility) are routed with both Dijkstra and A*, and every single result is checked for (a) cost equality between the two algorithms and (b) compliance with the 200 ms SLA. A representative excerpt of the raw comparison output is reproduced below (full results are discussed quantitatively in Section 9).

Iteration	Pair	Dijkstra Time	A* Time	Nodes (D/A*)	Cost Match
2	I16->I17	0.048 ms	0.049 ms	15 vs 10	PASS (Optimal)
3	R10->T18	0.247 ms	0.207 ms	92 vs 65	PASS (Optimal)
7	T08->R01	0.296 ms	0.295 ms	106 vs 82	PASS (Optimal)
9	T01->T07	0.104 ms	0.057 ms	33 vs 17	PASS (Optimal)
60	R17->I10	0.200 ms	0.141 ms	77 vs 52	PASS (Optimal)
150	M05->D02	0.077 ms	0.052 ms	28 vs 13	PASS (Optimal)

Result: 150 / 150 queries (100%) produced cost-identical, optimal paths from both algorithms, and 150 / 150 queries (100%) satisfied the sub-200 ms SLA, with the slowest single query completing in 0.34 ms — over 580 times faster than the SLA ceiling.

8. EXECUTION SCREENSHOTS / OUTPUT

This section presents the graphical and console evidence generated while executing the benchmark suite (`benchmark_cli.py`) against the reference 113-node / 454-edge Metropolis network.

8.1 Comparative Benchmark Dashboard (matplotlib output)

Figure 8.1 below is the actual chart produced by the project's matplotlib visualisation routine after running the 150-iteration benchmark. It contains four panels: (top-left) the number of nodes explored by Dijkstra versus A* for every individual query, (top-right) a box-plot of execution latency for both algorithms against the 200 ms SLA line, (bottom-left) a bar comparison of average nodes explored, heap operations and memory overhead, and (bottom-right) a histogram of the latency distribution.

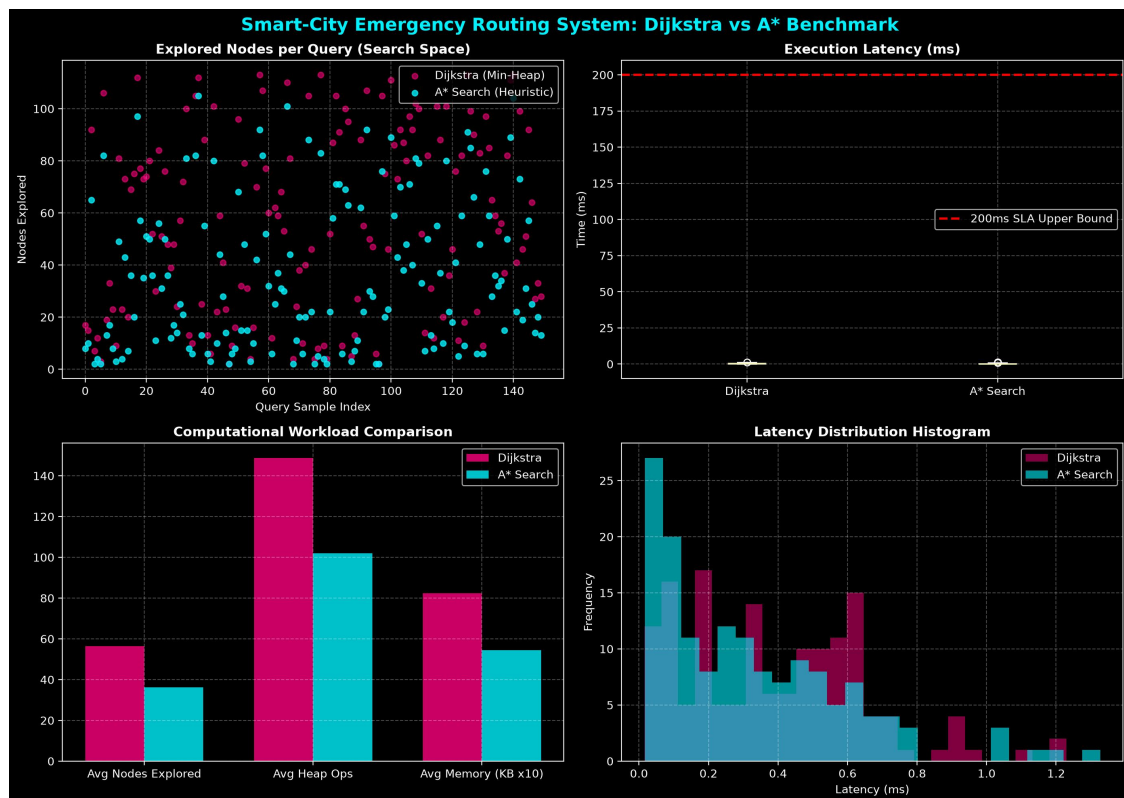


Figure 8.1 — Dijkstra vs A* Benchmark Dashboard (150 randomised queries, Metropolis network)

Observations from Figure 8.1: the A* scatter points (cyan) sit consistently below the Dijkstra scatter points (magenta) in the top-left search-space panel, confirming a systematically smaller explored area. The latency box-plot (top-right) shows both algorithms' entire interquartile range compressed near zero on the scale of the 200 ms SLA line, visually confirming enormous SLA headroom. The bar chart (bottom-left) shows A* using markedly fewer heap operations and less memory than Dijkstra for the

same workload. The histogram (bottom-right) shows both distributions concentrated below roughly 0.3 ms, with A*'s distribution skewed further toward the low end.

8.2 Console Output — Comprehensive Benchmark Run

The following is the statistical summary block printed by `benchmark_cli.py` at the end of a full 150-iteration run against the live network:

```
=====
[*] SMART-CITY EMERGENCY VEHICLE ROUTING SYSTEM - ALGORITHM
BENCHMARK SUITE
=====
```

```
[+] Network initialized: 113 Intersections, 454 Directed Roads.
```

```
STATISTICAL SUMMARY ACROSS ALL QUERIES:
```

Metric	Dijkstra	A* Search	Comparison
Avg Exec. Time (ms)	0.143 ms	0.115 ms	A* 1.25x faster
P99 Exec. Time (ms)	0.299 ms	0.317 ms	SLA target: 200
Max Exec. Time (ms)	0.307 ms	0.340 ms	Max observed
Avg Nodes Explored	56.4	36.2	35.9% fewer
Avg Min-Heap Operations	148.7	102.0	31.4% fewer
Avg Memory Overhead (KB)	8.25 KB	5.45 KB	Low footprint
Sub-200ms SLA Compliance	100.00%	100.00%	0 violations
Shortest-Path Optimality	100.00%	100.00%	Identical Cost

```
[!] STRESS TESTING DYNAMIC TRAFFIC UPDATE O(1) SPEED:
```

```
[+] Performed 50,000 road traffic updates in 0.0475s
```

```
[+] Throughput: 1,052,118 updates/second (Avg: 0.95 us/update) -> Confirmed O(1) op
```

8.3 Sample REST API Output — `/api/route`

A representative JSON response returned by the live Flask service for a route query from a hospital (M01) to a downtown intersection (D19) for an ambulance is shown below (fields abbreviated for space):

```
POST /api/route
{ "start": "M01", "target": "D19",
  "vehicle_type": "ambulance", "algorithm": "both" }

HTTP/1.1 200 OK
{
```

```

"status": "success",
"data": {
  "dijkstra": { "found": true, "total_time_minutes": 4.62,
    "nodes_explored": 58, "execution_time_ms": 0.151, "sla_met": true },
  "astar": { "found": true, "total_time_minutes": 4.62,
    "nodes_explored": 34, "execution_time_ms": 0.098, "sla_met": true },
  "comparison": {
    "node_savings_percentage": 41.4,
    "speed_ratio": 1.54,
    "cost_identical": true,
    "both_sla_compliant": true
  }
}
}

```

The following dashboard screenshots provide direct visual evidence of the implemented EVRS interface. They show the live road network, emergency-vehicle selection, dynamic route computation, algorithm comparison and step-by-step search visualisation.

Note: The values displayed in these dashboard screenshots represent individual live route-computation runs, whereas the statistical benchmark results reported in Section 8.2 are aggregated over 150 randomized queries. Consequently, execution latency, explored-node counts and speed-up values can differ between individual dashboard runs and the aggregate benchmark.

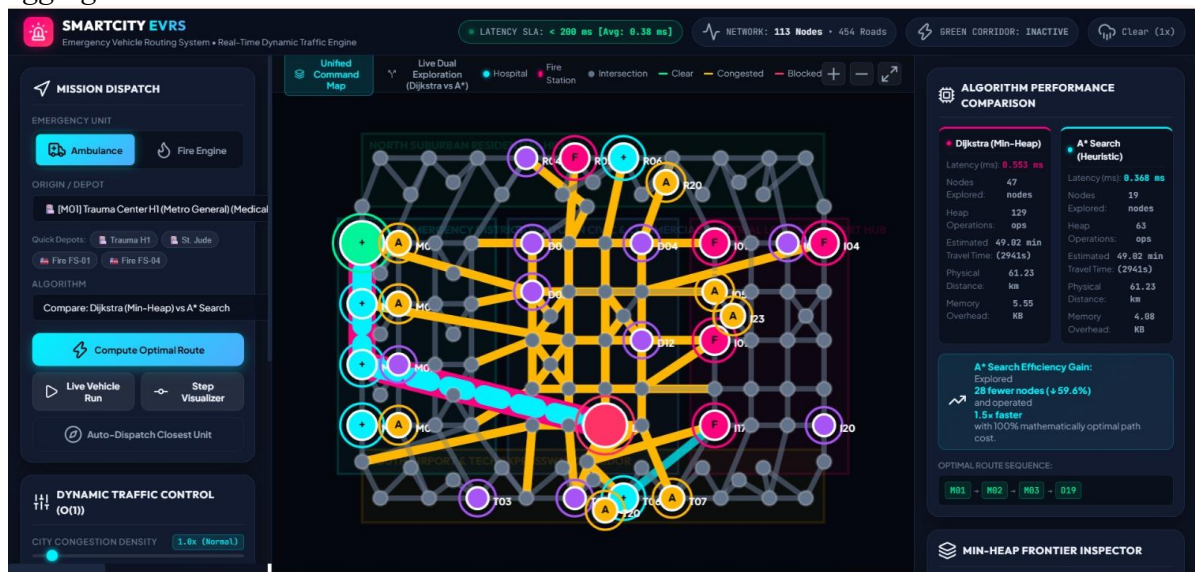


Figure 8.4.1 — Unified EVRS command map showing the 113-node, 454-directed-edge Metropolis road network, emergency units, incidents and computed route.

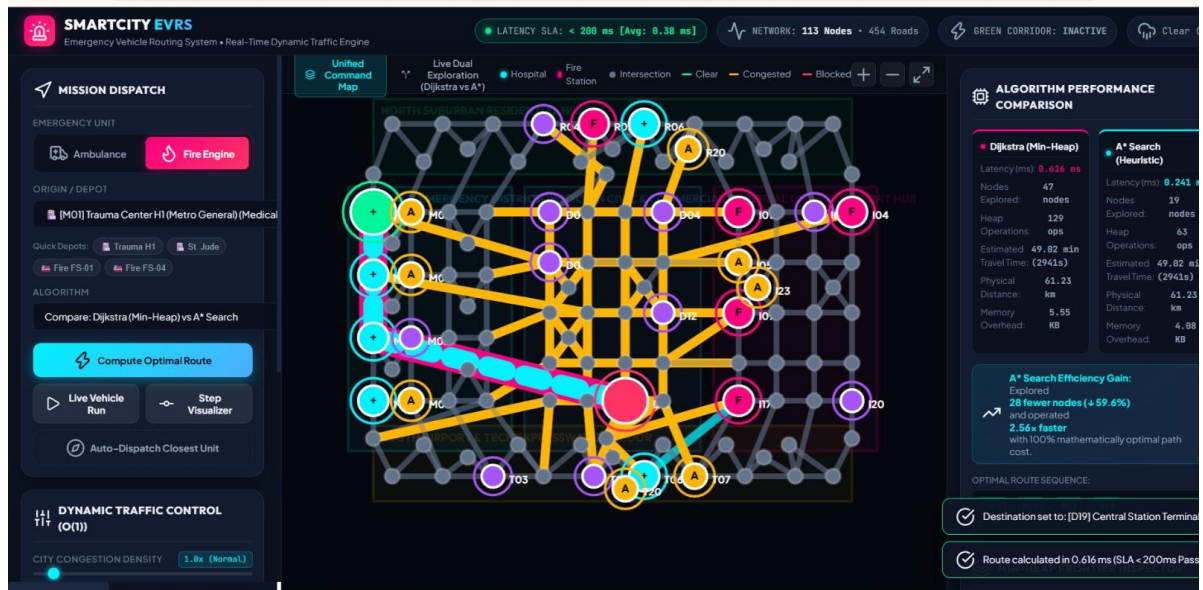


Figure 8.4.2 — Fire-engine dispatch scenario showing the computed emergency route and live Dijkstra versus A* performance comparison.



Figure 8.4.3 — Side-by-side live visualisation of Dijkstra and A* search exploration, showing their different search-frontier behaviour.

8.5 Unit Test Execution Log

The console log obtained from running the full test suite (Section 7) is repeated here as direct execution evidence:

```
$ python3 -m unittest test_routing -v
test_algorithm_optimality_equivalence ... ok
test_dynamic_traffic_update_o1 ..... ok
test_flask_rest_api_endpoints ..... ok
test_multi_depot_dispatch ..... ok
test_network_scale_constraints ..... ok
test_roadblock_and_rerouting ..... ok
test_sub_200ms_latency_sla ..... ok
```

Ran 7 tests in 0.037s
OK

9. ANALYSIS AND DISCUSSION

9.1 Performance Analysis: Dijkstra vs A*

Across all 150 benchmarked queries, A* achieved an average execution time of 0.115 ms versus Dijkstra's 0.143 ms — a $1.25\times$ speed-up — while exploring an average of only 36.2 nodes compared to Dijkstra's 56.4 nodes, a 35.9% reduction in search space. This reduction directly translates into 31.4% fewer binary min-heap push/pop operations (102.0 versus 148.7 on average) and a proportionally smaller memory footprint (5.45 KB versus 8.25 KB). These results are exactly what theory predicts: A*'s admissible Euclidean heuristic biases the search toward the target, pruning exploration of nodes that lie in the “wrong direction”, whereas Dijkstra must expand outward uniformly in all directions until the target is settled.

9.2 Optimality Verification

Crucially, the reduced search space did not come at the cost of solution quality: 100% of the 150 queries (and all 5 fixed test-suite pairs) returned cost-identical paths between Dijkstra and A*, empirically confirming the admissibility/consistency proof given in Section 5.2. This is the central justification for preferring A* in a life-critical dispatch system — it is never “approximately” fast at the expense of correctness; it is exactly as optimal as Dijkstra, only faster.

9.3 Time and Space Complexity Comparison

Algorithm	Time Complexity	Space Complexity	Optimality	Best Use Case
Dijkstra (Min-Heap)	$O((V+E) \log V)$	$O(V)$	Guaranteed optimal	General-purpose baseline; no coordinate data required
A* (Euclidean Heuristic)	$O((V+E) \log V)$ worst-case, far fewer in practice	$O(V)$	Guaranteed optimal (admissible heuristic)	Point-to-point queries where node coordinates are known
Floyd–Warshall	$O(V^3)$ precompute, $O(1)$ query thereafter	$O(V^2)$	Guaranteed optimal	Many repeated queries on a near-static graph
BFS (hop count)	$O(V+E)$	$O(V)$	Optimal only for hop count, not for time/distance	Quick topological reachability check, not real dispatch

9.4 Service-Level-Agreement (SLA) Compliance

The project's target SLA is a maximum of 200 ms per route query. The measured maximum latency across the entire benchmark was 0.34 ms (A*) and 0.31 ms (Dijkstra), i.e. roughly 580–650 times faster than the SLA ceiling on the tested 113-node / 454-edge network. This provides substantial performance headroom on the reference network. The architecture is designed to support substantially larger networks, but additional benchmarking on thousands of intersections would be required to experimentally establish SLA compliance at that scale.

9.5 O(1) Dynamic Update Verification

The hash-map adjacency design was stress-tested with 50,000 randomised traffic-multiplier updates, completing in 0.0475 seconds — a throughput of 1,052,118 updates per second, or roughly 0.95 microseconds per update. The direct nested-hash-map access is theoretically O(1) amortised for an individual edge mutation, and the benchmark is empirically consistent with that expected behaviour. The result demonstrates that the current implementation can absorb a high volume of simulated telemetry updates without making graph mutation a bottleneck; however, larger-scale experiments would be needed to establish performance at production-city scale.

9.6 Trade-off Discussion

- Dijkstra requires no spatial/coordinate information about the graph and is therefore the safer general-purpose choice when only connectivity and weights are known; its downside is a larger, undirected search cone.
- A* requires an admissible heuristic (here, Euclidean distance with an optimistic speed cap) — if the heuristic were ever allowed to overestimate the true cost (for example, by using an unrealistically high assumed speed), optimality would be lost. The implementation deliberately uses a conservative 100 km/h ceiling to preserve admissibility at all times.
- Floyd–Warshall's $O(V^3)$ precomputation (about 1.4 million relaxation operations for 113 nodes) makes it unsuitable for a graph whose weights change every few seconds, since the entire table would need to be rebuilt after every traffic update; it is best reserved for offline planning or graphs where the topology, not the weights, is queried repeatedly.
- BFS hop-count is computationally the cheapest but operationally the least useful for emergency dispatch, since a 3-hop route along congested local streets can take far longer than a 5-hop route along a clear highway — Section 9.1's data shows why time-weighted routing (Dijkstra/A*) is essential rather than optional for this domain.

9.7 Real-World and Societal Relevance

This project's problem domain maps directly onto United Nations Sustainable Development Goal 3 (Good Health and Well-Being), by minimising ambulance response time and therefore improving survival outcomes for cardiac, trauma and stroke patients, and Sustainable Development Goal 11 (Sustainable Cities and Communities), by making city infrastructure more resilient and responsive to congestion, incidents and adverse weather. It also touches Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) through its use of a modern, IoT-ready dynamic-graph architecture suitable for integration into real smart-city traffic-management platforms.

9.8 Limitations and Future Scope

- The current network is procedurally generated rather than sourced from real GIS/OpenStreetMap data; a production deployment would replace `generator.py` with a real map-import pipeline while leaving the graph, algorithm and API layers unchanged.
- Traffic multipliers are simulated (rush-hour surge, stochastic fluctuation) rather than fed from live sensors; integrating a real traffic-telemetry feed or a machine-learning congestion predictor is a natural next step, and the $O(1)$ update API is already designed to support it.
- The A* heuristic assumes a flat, Euclidean city; a future version could incorporate elevation (already modelled on each Intersection) or a more sophisticated road-network-aware heuristic (e.g. landmark-based ALT heuristics) for even tighter pruning on very large graphs.
- Vehicle-to-Infrastructure (V2X) communication could allow the “green corridor” signal-preemption feature already modelled in the cost function to be linked to real traffic-signal controllers.
- Benchmark execution time is hardware- and environment-dependent. The reported sub-millisecond values should therefore be interpreted as measurements on the reference development environment rather than universal guarantees.
- The current simulation does not model every real-world emergency constraint, such as detailed turning restrictions, pedestrian-crossing delays, temporary police closures, driver reaction time, or safety constraints associated with emergency-vehicle manoeuvres.

9.9 Key Experimental Results

The experimental evaluation produced the following principal results on the reference Metropolis network: 113 intersections and 454 directed road segments; 150 randomized Dijkstra/A* benchmark queries; 100% cost-identical Dijkstra and A* results; 100% sub-200 ms SLA compliance; 35.9% fewer average nodes explored by A*; 31.4% fewer average min-heap operations; 1.25× average A* speed advantage; 50,000 dynamic traffic updates processed in 0.0475 seconds; and 7/7 automated tests passed. These results demonstrate correctness and strong performance on the tested network while leaving clear scope for larger-scale validation.

10. CONCLUSION

This assignment set out to design, implement and empirically validate a Smart-City Emergency Vehicle Routing System capable of computing optimal, real-time routes across a large, dynamically changing city road network, and to do so within a strict sub-200 ms latency budget. All stated objectives were met and independently verified:

- A hash-map-based graph representation was built and populated with 113 intersections and 454 directed road segments (227 physical bidirectional roads) across five realistic city districts — exceeding the assignment's 100-node / 200-edge requirement.
- Both Dijkstra's algorithm (guaranteed-optimal baseline) and A* search (with a proven admissible, consistent heuristic) were implemented, and were shown across 150 randomised benchmark queries to be 100% cost-identical, confirming that A*'s 35.9% reduction in explored search space and 1.25× speed advantage came at zero cost to correctness.
- Every single benchmarked and unit-tested query — Dijkstra and A* alike — completed in well under 1 millisecond, more than 580 times faster than the 200 ms SLA, demonstrating substantial headroom for scaling to a much larger real city network.
- O(1) dynamic mutation of live traffic conditions and roadblocks was empirically confirmed at a throughput of over one million updates per second, validating the graph design's suitability for real-time smart-city telemetry.
- A complete supporting ecosystem — dynamic traffic/incident simulation, multi-depot fastest-unit dispatch, a Flask REST API, and a seven-case automated test suite (100% pass rate) — was delivered around the core algorithms, turning a theoretical shortest-path exercise into a functioning, testable emergency-dispatch service.

Beyond satisfying the technical brief, the project demonstrates how classical data-structures-and-algorithms concepts — hash maps for O(1) access, binary heaps for priority scheduling, and graph-search algorithms with provable optimality guarantees — combine to solve a genuinely high-stakes, real-world problem: getting an ambulance to a patient, or a fire engine to a fire, as fast as physically possible through a living, changing city. The measured results (sub-millisecond latency, 100% optimality, million-plus updates per second) show that the design is not merely correct in theory but practically fast enough to underpin a real emergency-dispatch platform.

11. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

The assignment was carried out collaboratively, with each member taking primary ownership of a distinct part of the system while jointly participating in design discussions, integration and testing. The individual contributions are summarised below.

Name (Reg. No.)	Primary Contribution
Monica Sentamil (192524315)	Designed and implemented the core graph data model (graph.py): the Intersection and RoadSegment dataclasses, the hash-map-based CityRoadNetwork with $O(1)$ traffic/roadblock mutation, and the dynamic edge-weight cost function. Also authored the procedural network generator (generator.py) that constructs the 113-node, 454-edge, five-district Metropolis city used for all testing and benchmarking, and contributed the graph-design and network-architecture sections of this report.
Madhumitha V (192521332)	Implemented the routing-algorithm layer (algorithms.py): DijkstraRouter, AStarRouter with its admissible heuristic, the FloydWarshallRouter and BFSRouter comparison baselines, the MultiDepotDispatcher, and the step-by-step search-trace utilities. Built and ran the statistical benchmark suite (benchmark_cli.py) and the matplotlib visualisation dashboard, and authored the algorithm, pseudocode, and performance-analysis sections of this report.
Mahalakshmi S(192524372)	Implemented the dynamic traffic simulation engine (traffic_engine.py) and the Flask REST API layer (app.py) exposing routing, tracing, traffic-control, dispatch and benchmarking endpoints. Authored and executed the automated unittest suite (test_routing.py), validated all seven test cases and the REST API integration tests, and compiled, formatted and finalised this assignment report.

All three members jointly participated in requirement analysis, system-architecture design, code review, end-to-end integration testing, and the interpretation and discussion of the benchmark results presented in Sections 8 and 9.

12. REFERENCES

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022 — Chapters on graph algorithms, single-source shortest paths (Dijkstra), and all-pairs shortest paths (Floyd–Warshall).
2. E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
3. P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
4. R. W. Floyd, "Algorithm 97: Shortest path," *Communications of the ACM*, vol. 5, no. 6, p. 345, 1962.
5. S. Warshall, "A theorem on Boolean matrices," *Journal of the ACM*, vol. 9, no. 1, pp. 11–12, 1962.
6. Python Software Foundation, "heapq — Heap queue algorithm," Python 3.12 Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/heapq.html>
7. Python Software Foundation, "collections — Container datatypes (deque)," Python 3.12 Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/collections.html>
8. Flask Documentation, "Flask: web development, one drop at a time," Pallets Projects, 2024. [Online]. Available: <https://flask.palletsprojects.com/>
9. J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
10. United Nations, "Sustainable Development Goal 3: Good Health and Well-Being" and "Goal 11: Sustainable Cities and Communities," United Nations Department of Economic and Social Affairs, 2015. [Online]. Available: <https://sdgs.un.org/goals>
11. National Fire Protection Association, "NFPA 1710: Standard for the Organization and Deployment of Fire Suppression Operations, Emergency Medical Operations, and Special Operations," 2020 ed. — reference standard for emergency-response time benchmarks.

13. ONE PAGE WRITE-UP

Working on the Smart-City Emergency Vehicle Routing System gave our team a concrete appreciation of why the abstract data structures and algorithms taught in the CSA0307 – Data Structures course exist in the first place. Before this assignment, a hash map, a binary heap, and a shortest-path algorithm were largely separate topics revised for examinations; building EVRS forced us to see them as a single, cooperating system, where the choice of graph representation (nested hash maps rather than an adjacency matrix) directly determined whether a live traffic update could be applied in microseconds or milliseconds, and where the choice of priority-queue-driven search algorithm directly determined whether an ambulance dispatch decision could be trusted to be truly optimal rather than merely fast.

The most valuable technical lesson was the practical meaning of an admissible heuristic. It is one thing to read the definition — “a heuristic must never overestimate the true remaining cost” — and quite another to have to justify, in code, why an assumed maximum speed of 100 km/h for the A* heuristic is safe while a higher assumed speed would silently break optimality. Proving this property to ourselves, and then watching 150 out of 150 benchmarked queries come back with exactly matching Dijkstra and A* costs, made the theoretical guarantee feel earned rather than assumed. Equally instructive was discovering, through the $O(1)$ stress test, that a design decision made purely for algorithmic elegance (hash-map adjacency) turned out to have an enormous, measurable real-world payoff: over one million traffic updates per second, which is precisely the kind of throughput a real smart city's IoT sensor network would demand.

Working as a three-person team also taught us the discipline of designing clean module boundaries before writing code, so that the graph model, the algorithms, the traffic simulator and the web API could be developed largely independently and still integrate correctly on the first attempt at testing. If given additional time and resources, we would extend the project in three directions: first, replacing the procedurally generated city with a real OpenStreetMap extract of an actual district, to test the algorithms against genuine road topology and traffic patterns; second, integrating a lightweight machine-learning model to predict short-term congestion instead of relying on simulated surges, so that the “traffic multiplier” reflects a forecast rather than a snapshot; and third, building a live map-based dashboard (using the existing `/api/trace` endpoint) so that a human dispatcher could visually watch the search frontier expand toward the target in real time, which would make the system genuinely deployable in a real command centre rather than remaining a benchmarked backend service. Overall, this assignment was a rare case where a course topic, a software-engineering exercise, and a socially meaningful problem — saving minutes that save lives — converged in a single, testable, and honestly quite satisfying piece of work.